



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE





Testes de Software: Tipos, Níveis e Cobertura

Uma visão formal e prática sobre como testar com qualidade, conectando a pirâmide de testes, cobertura de código e os critérios de aceitação Gherkin ao dia a dia de desenvolvimento.



O que vamos explorar hoje

01

Por que testar?

A motivação econômica e técnica por trás dos testes de software.

02

Tipos e Níveis de Teste

Unitário, integração, sistema e aceitação — cada um com seu papel.

03

Pirâmide de Testes

A lógica econômica por trás da forma da pirâmide.

04

Cobertura e Edge Cases

O que cobertura mede, suas limitações e como bons testes revelam bugs.

05

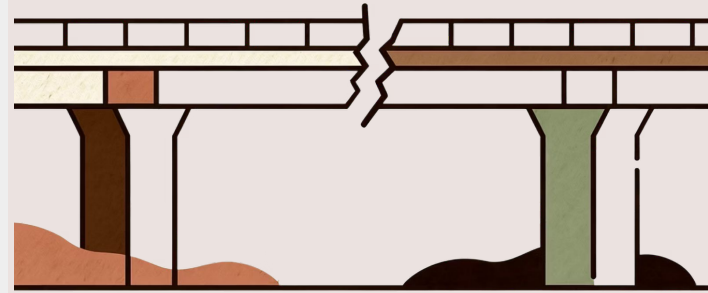
Conexão com Gherkin e DoD

Critérios de aceitação como base formal para testes de aceitação.



Por Que Testar?

Bugs descobertos em produção custam de **10 a 100 vezes mais** do que aqueles encontrados ainda em desenvolvimento. Cada defeito não detectado representa risco de negócio, perda de reputação e retrabalho caro. Testar não é um custo, é um investimento que reduz a incerteza e aumenta a confiança na entrega.



O Custo Cresce com o Tempo

Quanto mais tarde o bug é encontrado...

Maior o impacto: mais código foi escrito sobre a falha, mais usuários podem ter sido afetados, e mais difícil é rastrear a origem do problema.

A regra da amplificação do defeito

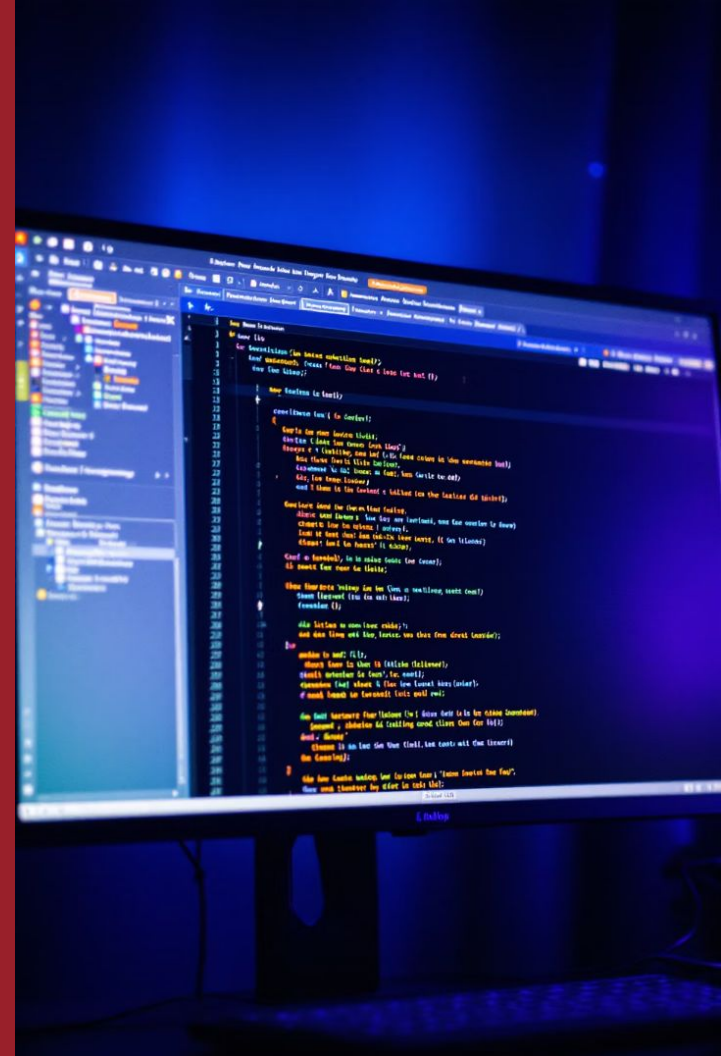
Um requisito mal compreendido vira design errado, que vira código errado, que vira teste errado — e chega ao usuário como um produto que não funciona. Quanto antes o problema é pego, menor o custo de correção.

- Requisito: custo = 1x
- Design: custo = 5x
- Código: custo = 10x
- Produção: custo = 100x





Evolução Histórica



Evolução Histórica dos Testes de Software

Era Inicial

Os primeiros testes de software, realizados na década de 1950, eram essencialmente manuais e focados em encontrar erros básicos. A depuração era uma atividade individual e, em muitos casos, ad hoc.

Década de 1970

O conceito de testes de sistema se consolidou, com foco em avaliar o comportamento do software todo. A qualidade do software passou a ser um critério central no processo de desenvolvimento.

Atualmente

Hoje, os testes de software englobam uma gama diversificada de metodologias e ferramentas automatizadas. A qualidade de software tornou-se um pilar fundamental para a competitividade no mercado.

1

2

3

4

5

Década de 1960

Com o surgimento de linguagens de programação mais complexas, as práticas de teste evoluíram. Testes de unidade e integração começaram a ganhar forma, para verificar o funcionamento individual de módulos de código.

Década de 1980

Testes de aceitação, focados na perspectiva do usuário final, surgiram como um novo tipo de teste para validar se o software atende às necessidades do cliente.



Fundamentos dos Testes de Software

Detecção de Defeitos

O objetivo primordial dos testes de software é identificar defeitos e falhas no código, prevenindo erros que podem afetar a funcionalidade e a qualidade do sistema.

Validação de Requisitos

Os testes garantem que o software atenda aos requisitos estabelecidos pela equipe de desenvolvimento e pelos stakeholders, garantindo que o produto final seja entregue conforme as expectativas.

Garantia de Qualidade

A implementação de práticas de teste contribui para a construção de software de alta qualidade, livre de erros e com melhor desempenho, proporcionando uma experiência positiva para o usuário.





Os Primeiros Anos dos Testes



Validação Manual

Testes realizados
manualmente pelos
próprios desenvolvedores.



Foco em Erros Sintáticos

Principal preocupação era
corrigir erros básicos de
sintaxe.



Falta de Processos

Ausência de metodologias estruturadas levava a falhas não
detectadas.



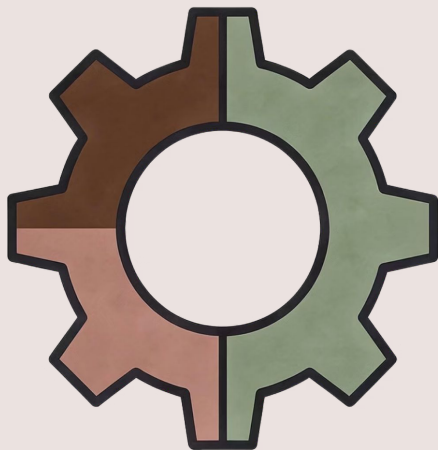


Tipos de Teste

Cada nível da cadeia de desenvolvimento exige um tipo diferente de verificação — do detalhe mais fino ao comportamento completo do sistema.



Teste Unitário



O que é?

Testa a menor unidade isolável do código (geralmente uma função ou método) verificando se ela se comporta corretamente para diversas entradas.

Características

- Rápido de executar (milissegundos)
- Isolado de dependências externas
- Deve ser determinístico
- Escrito pelo próprio desenvolvedor

Exemplo prático

Uma função `calcularDesconto(preco, percentual)` deve retornar o valor correto para entradas válidas, lançar exceção para percentuais negativos e tratar corretamente o caso de preço zero. Cada um desses cenários é um teste unitário.

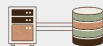


Teste de Integração



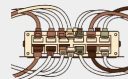
O que verifica?

A interação entre dois ou mais módulos. Não basta que cada peça funcione sozinha — é preciso garantir que elas se comunicam corretamente quando conectadas.



Exemplos comuns

Integração entre serviço e banco de dados, chamada entre microserviços, consumo de uma API externa ou comunicação entre camadas (controller → service → repository).



Atenção ao escopo

Testes de integração são mais lentos e frágeis que os unitários. Use-os para verificar contratos entre componentes, não para testar lógica interna de cada módulo.



Teste de Sistema

O teste de sistema valida o comportamento do **software como um todo**, exercitando fluxos completos de ponta a ponta — do input do usuário até a resposta final do sistema. Inclui testes funcionais, de performance, segurança e usabilidade.

Funcional

O sistema faz o que foi especificado?

Performance

Responde dentro do tempo aceitável sob carga?

Segurança

Resiste a ataques e acessos indevidos?

Usabilidade

O usuário consegue operar o sistema com facilidade?



Teste de Aceitação

O teste de aceitação responde à pergunta mais importante: "O sistema entrega o valor esperado pelo usuário e pelo negócio?" É realizado do ponto de vista do usuário final ou do product owner, verificando se os critérios de aceitação definidos foram atendidos.

Dois tipos principais

- **UAT (User Acceptance Testing):** conduzido pelo cliente ou usuário final em ambiente próximo ao de produção
- **ATDD (Acceptance Test-Driven Development):** critérios escritos antes do código, guiando o desenvolvimento

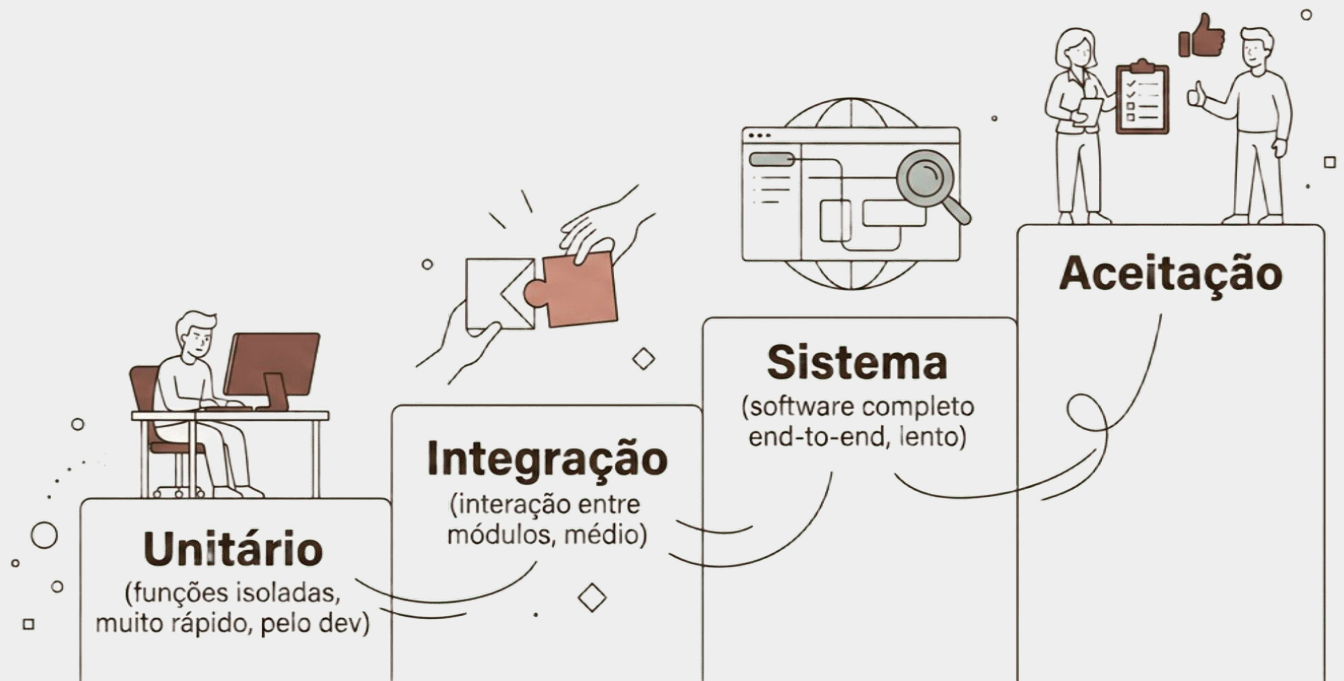
Pergunta-chave

Um software pode passar em todos os testes unitários e de integração e ainda assim não atender ao que o usuário precisava. O teste de aceitação é a última linha de defesa antes da entrega.



Os Quatro Níveis em Perspectiva

Cada nível tem escopo, velocidade e responsável diferente. Uma boa estratégia de testes usa todos os quatro de forma complementar.



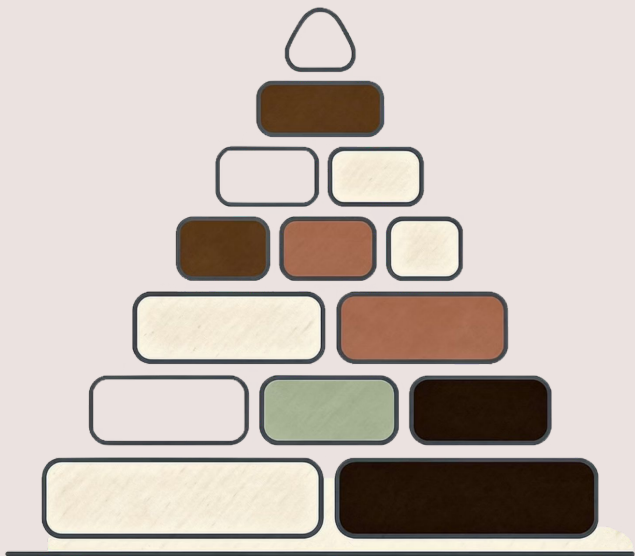


A Pirâmide de Testes

Uma metáfora visual que esconde uma lógica econômica poderosa sobre como distribuir esforço e investimento em testes.



A Forma da Pirâmide



A pirâmide de testes, proposta por Mike Cohn, representa a **distribuição ideal** de testes em um projeto: **muitos testes unitários na base**, uma camada intermediária de testes de integração, e poucos testes de sistema e aceitação no topo. Essa forma não é arbitrária: ela reflete o custo, a velocidade e o valor de cada tipo de teste.



A Lógica Econômica da Pirâmide



Base larga = testes rápidos e baratos

Testes unitários rodam em milissegundos, não precisam de infraestrutura e são fáceis de manter. Centenas ou milhares deles são viáveis e recomendados.



Meio = equilíbrio entre cobertura e custo

Testes de integração verificam contratos críticos entre componentes. São mais lentos, mas essenciais para garantir que o sistema se monta corretamente.



Topo estreito = testes lentos e caros

Testes end-to-end e de aceitação são valiosos mas custosos. Devem cobrir apenas os fluxos críticos, não substituem os níveis inferiores.



O Anti-Padrão: O Sorvete

⚠ O que é o anti-padrão do sorvete?

Quando uma equipe inverte a pirâmide — muitos testes manuais e end-to-end no topo, poucos ou nenhum teste unitário na base — ela cria uma suíte de testes frágil, lenta e cara de manter.

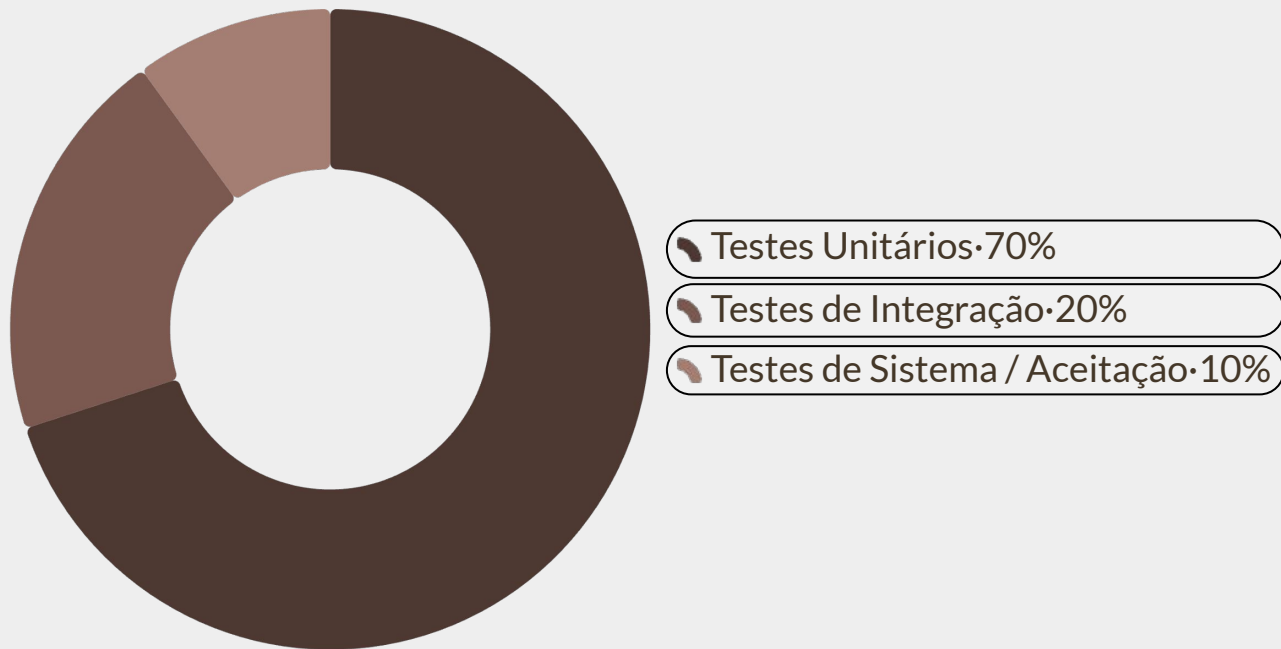
Consequências práticas

- Pipeline de CI/CD lento (horas para rodar)
- Testes que falham por razões aleatórias (flaky tests)
- Difícil identificar a causa raiz de uma falha
- Alta dependência de ambientes de staging
- Feedback tardio para o desenvolvedor

O objetivo é ter o **feedback mais rápido possível** no nível mais baixo possível.



Proporção Recomendada na Pirâmide



A proporção 70/20/10 é uma referência amplamente adotada. Projetos com muita lógica de negócio podem ter proporções ainda mais favoráveis aos testes unitários. O importante é que a base seja sólida.





Cobertura de Testes

Uma métrica poderosa e frequentemente mal interpretada. Entender o que ela mede (e o que não mede) é essencial para usá-la bem.



O Que É Cobertura de Código?

Cobertura de código (code coverage) mede a porcentagem de linhas, branches ou caminhos do código que são exercitados pelos testes. É uma métrica de verificação: ela diz que o código foi *executado* durante os testes, mas não garante que o comportamento foi *verificado corretamente*.



Tipos de Cobertura

Cobertura de Linhas

Percentual de linhas de código executadas. O tipo mais simples e mais comum. Uma linha "coberta" apenas significa que foi rodada, não que foi testada de forma significativa.

Cobertura de Branches

Verifica se cada ramificação de um `if/else` foi exercitada, tanto o caminho verdadeiro quanto o falso. Mais completa que cobertura de linhas.

Cobertura de Caminhos

Garante que todas as combinações possíveis de caminhos foram testadas. É a mais rigorosa, mas pode ser impraticável em código complexo com muitas condições aninhadas.



Limitações da Cobertura

⚠ 100% de cobertura não significa ausência de bugs. Significa apenas que todas as linhas foram executadas — não que todos os comportamentos foram corretamente verificados.

O que cobertura NÃO garante

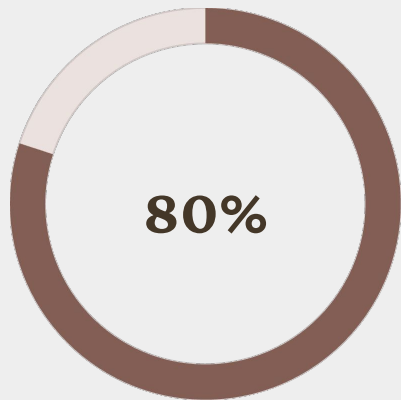
- Que os assertions são corretos
- Que edge cases foram testados
- Que o comportamento está correto do ponto de vista do negócio
- Que integrações funcionam

Cobertura como guia, não como meta

Use cobertura para identificar código **não testado** e priorizar onde escrever novos testes. Evite transformá-la em meta absoluta — equipes que perseguem 100% tendem a escrever testes sem valor real.

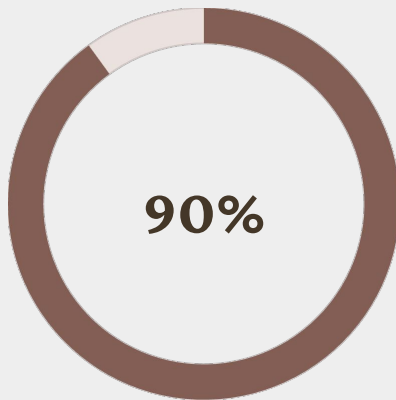


Referências de Cobertura no Mercado



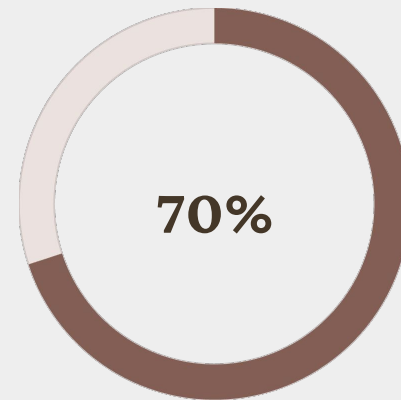
Cobertura mínima recomendada

Ponto de partida saudável para a maioria dos projetos de negócio.



Alta cobertura

Esperado em sistemas críticos como fintech, saúde e infraestrutura.



Cobertura de branches

Meta prática para cobertura de branches em projetos com lógica condicional complexa.





Edge Cases & Bugs

Bons testes não apenas verificam o caminho feliz, eles exploram os limites e casos extremos onde os bugs costumam se esconder.



O Que São Edge Cases?

Edge cases (casos extremos ou de borda) são situações que ocorrem nos limites dos intervalos de entrada válidos, ou fora deles. São exatamente onde a maioria dos bugs se esconde, porque os desenvolvedores tendem a pensar no "caminho feliz" e negligenciar os extremos.



Exemplos de Edge Cases Clássicos



Valores nulos e zero

O que acontece quando um campo obrigatório é null? Divisão por zero? Lista vazia? Esses casos frequentemente causam `NullPointerException` ou comportamentos inesperados.



Limites de intervalo

O valor máximo de um inteiro, uma data no último dia do mês, um campo de texto com exatamente o limite de caracteres, tudo isso são edge cases críticos a testar.

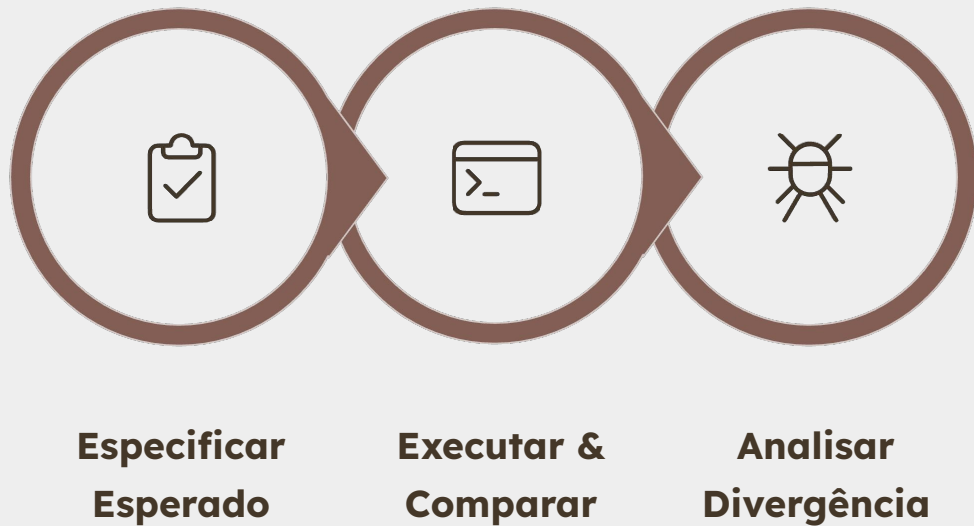


Concorrência e estado

Dois usuários modificando o mesmo registro ao mesmo tempo, um processo interrompido no meio: race conditions e problemas de estado são edge cases difíceis, mas frequentes em sistemas distribuídos.



Como Bons Testes Revelam Bugs



Um teste que falha não é um problema, é informação valiosa. Ele indica exatamente onde o comportamento real diverge do esperado, acelerando o diagnóstico e a correção.



A Relação Entre Testes e Qualidade

"O objetivo de testar não é mostrar que o programa funciona, é encontrar os casos onde ele não funciona." — Edsger W. Dijkstra

Mudar a mentalidade de "provar que funciona" para "tentar quebrar" é o que transforma um teste superficial em um teste eficaz. Edge cases, dados inválidos, sequências inesperadas de ações, tudo isso deve fazer parte do repertório de um bom tester.





Gherkin & Definition of Done

Como os critérios de aceitação escritos em Gherkin se transformam em testes formais, e como o DoD garante que "pronto" realmente significa pronto.



Relembrando o Gherkin

A estrutura Given-When-Then

O Gherkin é uma linguagem de especificação legível por humanos que descreve o comportamento esperado de uma feature em termos de **contexto, ação e resultado**. É a ponte entre o que o negócio quer e o que o sistema deve fazer.

Exemplo

Funcionalidade: Login de usuário

Cenário: Login com credenciais válidas

Dado que o usuário está na tela de login

Quando ele inserir email e senha corretos

Então deve ser redirecionado ao dashboard



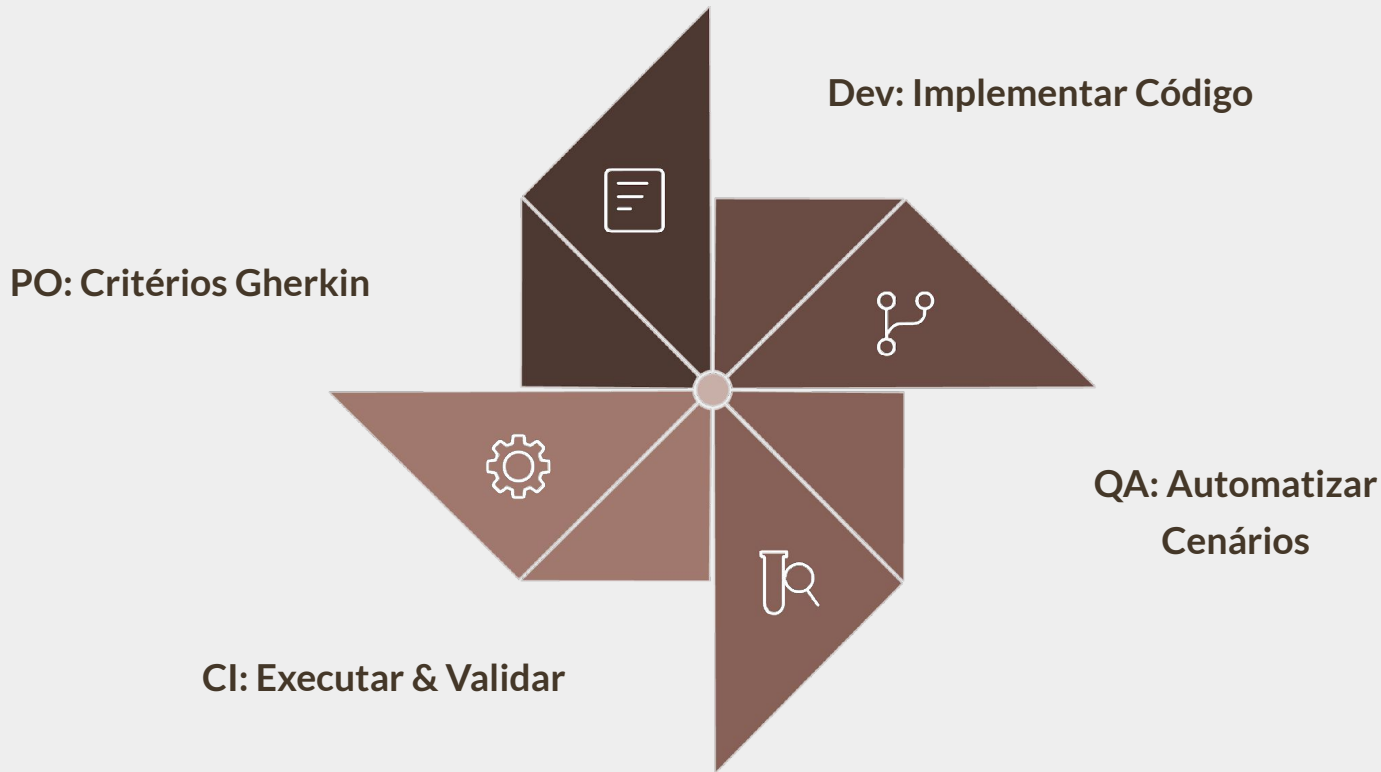
Gherkin Como Contrato de Teste

Cada cenário Gherkin é, na prática, um **caso de teste de aceitação** já especificado. O *Given* define a pré-condição (setup do teste), o *When* define a ação (execução), e o *Then* define o resultado esperado (assertion). Ferramentas como **Cucumber**, **SpecFlow** e **Behave** transformam esses cenários em testes automatizados executáveis.



ATDD: O Ciclo Completo

No ATDD (Acceptance Test-Driven Development), os critérios Gherkin são escritos *antes* do código. Isso cria um alinhamento real entre PO, Dev e QA desde o início da história.



Definition of Done e os Testes

O que é o DoD?

O **Definition of Done** é um acordo da equipe que define o que precisa estar completo para que uma história seja considerada "pronta". Ele é a garantia de qualidade mínima aceita pela equipe.

Testes como critério de DoD

Um DoD bem definido inclui: testes unitários escritos e passando, cobertura mínima atingida, critérios Gherkin automatizados, sem regressões no pipeline de CI/CD.

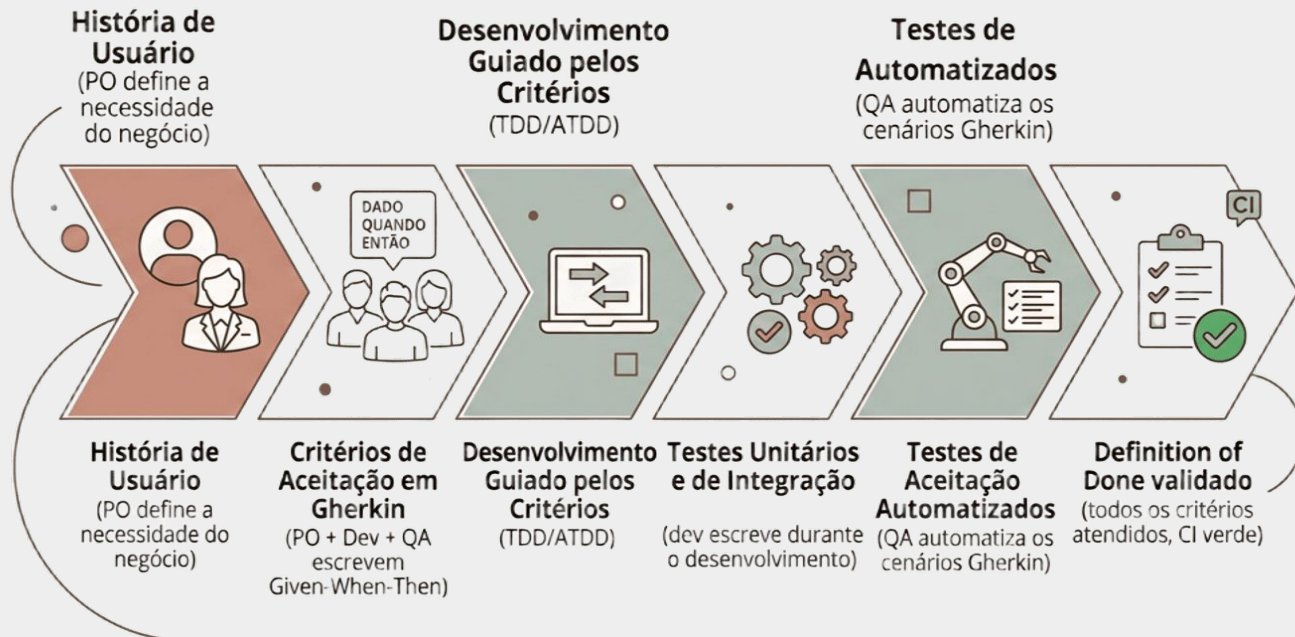
DoD + Gherkin = qualidade auditável

Quando os critérios de aceitação Gherkin fazem parte do DoD, a equipe tem uma definição clara, verificável e rastreável do que significa entregar valor de forma responsável.



Conectando Tudo: Da História ao Teste

Este fluxo garante rastreabilidade completa: cada linha de código existe para satisfazer um critério de aceitação, e cada critério é verificado por um teste automatizado.



Exemplo Completo: Do Gherkin ao Código

Cenário Gherkin

Funcionalidade: Cálculo de desconto

Cenário: Desconto para cliente VIP

Dado que o cliente tem status VIP

E o valor da compra é R\$ 200,00

Quando o desconto for calculado

Então o valor final deve ser R\$

160,00

E o desconto aplicado deve ser 20%

Teste Unitário Correspondente

```
@Test
```

```
void
```

```
clienteVIP_deveReceberDescontoDe20() {
```

```
    Cliente c = new Cliente(Status.VIP);
```

```
    Pedido p = new Pedido(c, 200.0);
```

```
    ResultadoDesconto r =
```

```
    descontoService.calcular(p);
```

```
    assertEquals(160.0,
```

```
    r.getValorFinal());
```

```
    assertEquals(0.20, r.getPercentual());
```

```
}
```



Boas Práticas: O Teste F.I.R.S.T.

Fast

Testes devem rodar rapidamente. Testes lentos são evitados e acumulam dívida técnica.



Independent

Cada teste deve ser independente, não depender de outros testes nem de ordem de execução.



Repeatable

O mesmo teste deve produzir o mesmo resultado em qualquer ambiente e momento.

Self-Validating

O teste deve retornar pass ou fail, sem interpretação manual do resultado.



Timely

Testes devem ser escritos junto com (ou antes do) código de produção, não depois.



Ferramentas por Nível de Teste

Nível	Java / Kotlin	JavaScript / TS	Python
Unitário	JUnit 5, Mockito	Jest, Vitest	pytest, unittest
Integração	Spring Boot Test, Testcontainers	Supertest, Testcontainers	pytest + SQLAlchemy
Sistema / E2E	Selenium, Playwright	Playwright, Cypress	Playwright, Robot
Aceitação / Gherkin	Cucumber JVM	Cucumber.js	Behave, pytest-bdd

Escolha ferramentas que se integrem bem ao seu pipeline de CI/CD. A automação é o que transforma testes em qualidade contínua.



Integrando Testes ao Pipeline de CI/CD

De nada adianta ter uma suíte de testes robusta se ela não rodar automaticamente a cada commit. O pipeline de CI/CD é o guardião da qualidade, ele executa os testes, mede a cobertura e bloqueia merges que quebram o padrão definido no DoD.



Síntese: Os Cinco Pilares do Teste Eficaz



Pirâmide equilibrada

Muitos testes unitários, integração seletiva, poucos E2E, mantendo velocidade e cobertura.



Gherkin como especificação

Critérios de aceitação em Given-When-Then são contratos verificáveis que alinham negócio e tecnologia.



Edge cases cobertos

Pensar além do caminho feliz é o que diferencia testes mediocres de testes que realmente encontram bugs.



DoD como garantia

Definition of Done com critérios de teste garante que "pronto" tem o mesmo significado para todos da equipe.



Cobertura como guia

Use cobertura para encontrar brechas, não como meta absoluta. Qualidade importa mais que número.





E no projeto?

1 Audite sua pirâmide atual

Quantos testes unitários, de integração e E2E seu projeto tem? A proporção está saudável?

2 Revise seu DoD

Seu Definition of Done inclui critérios de cobertura e automação de testes de aceitação?

3 Escreva um cenário Gherkin

Escolha uma história do backlog atual e escreva seus critérios de aceitação em Given-When-Then.

4 Automatize um edge case

Identifique uma função crítica e escreva pelo menos dois testes de borda que ainda não existem.



Resumo da Aula

O que aprendemos

- Testes reduzem o custo de defeitos e aumentam a confiança na entrega
- Unitário → Integração → Sistema → Aceitação: cada nível tem escopo e responsabilidade distintos
- A pirâmide reflete uma lógica econômica: testes baratos e rápidos na base
- Cobertura mede execução, não correção, use como guia
- Edge cases são onde os bugs se escondem: teste os extremos
- Gherkin transforma critérios de aceitação em testes formais e rastreáveis
- DoD com critérios de teste garante qualidade auditável na entrega

A grande ideia

Testar não é uma fase separada do desenvolvimento, é uma prática contínua e integrada que começa na especificação (Gherkin), atravessa o desenvolvimento (TDD) e termina na validação automatizada (CI/CD). Qualidade é construída, não verificada.



Para amanhã

Testes de aceitação + BDD + Playwright

Conteúdo:  Leitura: Cap. 9, seção 9.2.4 (Testes de Aceitação) + visitar Cap. 7

Artefato: backlog com critérios Gherkin da Semana 4

Amanhã: BDD, cenários formais e demonstração do Playwright



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
[viniciusgarcia.me](https://www.viniciusgarcia.me)

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Testes de Software: Tipos, Níveis
e Cobertura